YOTSUBA SHADER

Referencia de sintaxis

Cada concepto del lenguaje en las sintaxis HLSL, GLSL, MSL, SPIR-V y WGSL

Edición 1.0 · octubre de 2026 · Generada a partir de los fuentes de atributos de los proyectos YotsubaShader.<X>Syntax

Yotsuba Shader te deja escribir el mismo shader con los nombres de cinco lenguajes de shaders. Cada conjunto de nombres es una **sintaxis**, que se distribuye como un paquete propio con un namespace del mismo nombre. Todas se asocian a los mismos conceptos, así que el compilador produce la misma representación intermedia uses la que uses. La sintaxis nunca decide la salida: los destinos de compilación se eligen por separado, en la propiedad `<YotsubaTargets>` de tu proyecto.

Los cinco paquetes de sintaxis

Sintaxis	Paquete y namespace	Namespace de recursos	Inspirada en	Atributos
HLSL	`YotsubaShader.HlslSyntax`	`YotsubaShader.HlslSyntax.Resources`	HLSL (Direct3D)	24
GLSL	`YotsubaShader.GlslSyntax`	`YotsubaShader.GlslSyntax.Resources`	GLSL (OpenGL / GLSL de Vulkan)	34
MSL	`YotsubaShader.MslSyntax`	`YotsubaShader.MslSyntax.Resources`	Metal Shading Language	30
SPIR-V	`YotsubaShader.SpirvSyntax`	`YotsubaShader.SpirvSyntax.Resources`	SPIR-V (Vulkan)	33
WGSL	`YotsubaShader.WgslSyntax`	`YotsubaShader.WgslSyntax.Resources`	WGSL (WebGPU)	23

Núcleo neutro: YotsubaShader.Language

Estos atributos son los mismos en todas las sintaxis. Los motores y frameworks se apoyan en este núcleo; cada paquete de sintaxis solo añade nombres.

Concepto	Atributo
Ubicación de interfaz (entradas de vértice, varyings, salidas de fragmento)	`[Location(n)]`
Uniform buffer (forma neutra)	`[Uniform]`
Storage buffer (forma neutra)	`[StorageBuffer]`
Set y binding (forma neutra)	`[Binding(set, binding)]`
Etapa de shader de un método	`[ShaderStage(Stage.Vertex | Fragment | Compute)]`

Valores integrados

Concepto	HLSL	GLSL	MSL	SPIR-V	WGSL
Etapa vertex					
Posición en clip space (salida del vertex)	`SV_Position`	`gl_Position`	`position`	`BuiltIn_Position`	`@builtin_position`
Índice de vértice	`SV_VertexID`	`gl_VertexIndex`	`vertex_id`	`BuiltIn_VertexIndex`	`@builtin_vertex_index`
Índice de instancia	`SV_InstanceID`	`gl_InstanceIndex`	`instance_id`	`BuiltIn_InstanceIndex`	`@builtin_instance_index`
Vértice base	`SV_StartVertexLocation`	`gl_BaseVertex`	`base_vertex`	`BuiltIn_BaseVertex`	—

Concepto	HLSL	GLSL	MSL	SPIR-V	WGSL
Instancia base	`SV_StartInstanceLocation`	`gl_BaseInstance`	`base_instance`	`BuiltIn_BaseInstance`	—
Distancia de recorte	`SV_ClipDistanceN`	`gl_ClipDistance`	`clip_distance`	`BuiltIn_ClipDistance`	`@builtin_clip_distances`
Capa del render target	`SV_RenderTargetArrayIndex`	`gl_Layer`	`render_target_array_index`	`BuiltIn_Layer`	—
Índice de viewport	`SV_ViewportArrayIndex`	`gl_ViewportIndex`	`viewport_array_index`	`BuiltIn_ViewportIndex`	—
Etapa fragment					
Coordenada del fragmento	`SV_Position`	`gl_FragCoord`	`position`	`BuiltIn_FragCoord`	`@builtin_position`
Indicador de cara frontal	`SV_IsFrontFace`	`gl_FrontFacing`	`front_facing`	`BuiltIn_FrontFacing`	`@builtin_front_facing`
Profundidad del fragmento (salida)	`SV_Depth`	`gl_FragDepth`	`depth`	`BuiltIn_FragDepth`	`@builtin_frag_depth`
Profundidad conservadora	`SV_DepthGreaterEqual` `SV_DepthLessEqual`	—	`depth("greater")` `depth("less")`	—	—
Referencia de stencil (salida)	`SV_StencilRef`	`gl_FragStencilRefARB`	`stencil`	`BuiltIn_FragStencilRef`	—
Índice de muestra	`SV_SampleIndex`	`gl_SampleID`	`sample_id`	`BuiltIn_SampleId`	`@builtin_sample_index`
Máscara de muestras	`SV_Coverage`	`gl_SampleMask` `gl_SampleMaskIn`	`sample_mask`	`BuiltIn_SampleMask`	`@builtin_sample_mask`
Índice de primitiva	`SV_PrimitiveID`	`gl_PrimitiveID`	`primitive_id`	`BuiltIn_PrimitiveId`	`@builtin_primitive_index`
Coordenadas baricéntricas	`SV_Barycentrics`	`gl_BaryCoordEXT`	`barycentric_coord`	`BuiltIn_BaryCoord`	—
Coordenadas baricéntricas (sin perspectiva)	—	`gl_BaryCoordNoPerspEXT`	—	`BuiltIn_BaryCoordNoPersp`	—
Invocación auxiliar	—	`gl_HelperInvocation`	—	`BuiltIn_HelperInvocation`	—
Posición de la muestra	—	`gl_SamplePosition`	—	`BuiltIn_SamplePosition`	—
Etapa compute					
Id de invocación global	`SV_DispatchThreadID`	`gl_GlobalInvocationID`	`thread_position_in_grid`	`BuiltIn_GlobalInvocationId`	`@builtin_global_invocation_id`
Id de invocación local	`SV_GroupThreadID`	`gl_LocalInvocationID`	`thread_position_in_threadgroup`	`BuiltIn_LocalInvocationId`	`@builtin_local_invocation_id`
Índice de invocación local	`SV_GroupIndex`	`gl_LocalInvocationIndex`	`thread_index_in_threadgroup`	`BuiltIn_LocalInvocationIndex`	`@builtin_local_invocation_index`
Id del grupo de trabajo	`SV_GroupID`	`gl_WorkGroupID`	`threadgroup_position_in_grid`	`BuiltIn_WorkgroupId`	`@builtin_workgroup_id`
Número de grupos de trabajo	—	`gl_NumWorkGroups`	`threadgroups_per_grid`	`BuiltIn_NumWorkgroups`	`@builtin_num_workgroups`
Tamaño del grupo (atributo de método)	`numthreads`	`layout_local_size`	—	—	`@workgroup_size`
Tamaño del grupo (valor integrado)	—	`gl_WorkGroupSize`	`threads_per_threadgroup`	`BuiltIn_WorkgroupSize`	—
Tamaño del subgrupo	—	`gl_SubgroupSize`	`threads_per_simdgroup`	`BuiltIn_SubgroupSize`	`@builtin_subgroup_size`
Id de invocación del subgrupo	—	`gl_SubgroupInvocationID`	`thread_index_in_simdgroup`	`BuiltIn_SubgroupInvocationId`	`@builtin_subgroup_invocation_id`
Id del subgrupo	—	`gl_SubgroupID`	`simdgroup_index_in_threadgroup`	`BuiltIn_SubgroupId`	`@builtin_subgroup_id`
Número de subgrupos	—	`gl_NumSubgroups`	`simdgroups_per_threadgroup`	`BuiltIn_NumSubgroups`	`@builtin_num_subgroups`

Recursos y slots de binding

Concepto	HLSL	GLSL	MSL	SPIR-V	WGSL
Uniform buffer	`cbuffer, ConstantBuffer<T>`	`uniform`	`constant`	`Uniform`	`uniform`
Storage buffer de lectura y escritura	`RWStructuredBuffer<T>`	`buffer`	`device`	`StorageBuffer`	`storage(storage_use.read_write)`
Storage buffer de solo lectura	`StructuredBuffer<T>`	`buffer`	`device(readOnly: true)`	`StorageBuffer`	`storage(storage_use.read)`

Concepto	HLSL	GLSL	MSL	SPIR-V	WGSL
Slot de binding	`register(RegisterPrefix.b, n)`	`layout(set, binding)`	`buffer(n)`, `texture(n)`, `sampler(n)`	`DescriptorSet(s)` + `Binding(n)`	`@group(s)` + `@binding(n)`
Slot de textura	`register(RegisterPrefix.t, n)`	`layout(set, binding)`	`texture(n)`	`DescriptorSet(s)` + `Binding(n)`	`@group(s)` + `@binding(n)`
Slot de sampler	`register(RegisterPrefix.s, n)`	`layout(set, binding)`	`sampler(n)`	`DescriptorSet(s)` + `Binding(n)`	`@group(s)` + `@binding(n)`

Notas

- **Posición y coordenada del fragmento.** `SV_Position`, `position` y `@builtin_position` son un solo atributo para ambas: en una salida del vertex es la posición en clip space y en una entrada del fragment es la coordenada del fragmento. GLSL y SPIR-V tienen un nombre para cada una (`gl_Position` / `gl_FragCoord`, `BuiltIn_Position` / `BuiltIn_FragCoord`).
- **Tamaño del grupo de trabajo.** HLSL, GLSL y WGSL lo declaran con un atributo de método (`numthreads`, `layout_local_size`, `@workgroup_size`). MSL y SPIR-V lo exponen como un valor integrado que el shader lee.
- **Varias formas en una celda** (por ejemplo `gl_SampleMask` / `gl_SampleMaskIn`) son la forma de salida y la de entrada del mismo concepto, o variantes equivalentes.
- **Una sintaxis por archivo.** Algunos nombres existen en más de una sintaxis (`uniform`, `buffer`). Importa un namespace de sintaxis por archivo para que no sean ambiguos.
- **Qué backend admite cada concepto** está en el PDF complementario, *Compatibilidad de backends*.